

Ejercicio1: Construcción manual de una red neuronal de clasificación

Vamos a construir una red neuronal de clasificación con función de activación sigmoide. La arquitectura de la red consta de:

- **Entradas:** 2 características por muestra.
- **Neurona única:** con un vector de pesos y un sesgo.
- **Función de activación:** Sigmoide.
- **Función de pérdida:** Error cuadrático medio (MSE).

Las entradas están dadas por la matriz:

$$X = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 0 \\ 1 & 1 \\ 0 & 0 \\ 1 & 0 \\ 2 & 2 \\ 3 & 1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$$

Los pesos y sesgo iniciales para la neurona son:

$$\mathbf{w} = [0, 1, 0, 2], \quad b = 0, 1.$$

Las etiquetas reales (targets) son:

$$Y = [0, 1, 1, 1, 1, 1, 0, 0, 0, 1].$$

La tasa de aprendizaje es $\eta = 0,01$.

Parte 1: Cálculo manual para una muestra

Para una sola muestra de entrada $\mathbf{x} = [x_1, x_2]$ y etiqueta real y , con pesos $\mathbf{w} = [w_1, w_2]$ y sesgo b , se realiza lo siguiente:

Paso 1: Calcula la salida lineal de la neurona

$$z = w_1 x_1 + w_2 x_2 + b.$$

Paso 2: Calcula la salida después de la función de activación sigmoide

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Paso 3: Calcula la función de pérdida (MSE) para la salida obtenida

$$J = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2} \left(\frac{1}{1 + e^{-z}} - y \right)^2.$$

Para simplificar, puedes trabajar con el error de cada neurona:

$$e^{(k)} = \hat{y}^{(k)} - y^{(k)}.$$

Paso 4: Calcula el gradiente de la pérdida respecto a los pesos y el sesgo aplicando la regla de la cadena
En este paso, debes calcular cómo cambia la función de pérdida J con respecto a los parámetros del modelo: los pesos w_1 , w_2 y el sesgo b . Para ello, **aplica explícitamente la regla de la cadena paso a paso**.

Parte A: Derivada de la función sigmoide

Primero, demuestra que la derivada de la función sigmoide $\sigma(z)$ está dada por:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \Rightarrow \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Parte B: Aplicación de la regla de la cadena

Sabemos que:

- La función de activación es $\hat{y} = \sigma(z)$, donde $z = w_1x_1 + w_2x_2 + b$.
- La función de pérdida es $J = \frac{1}{2}(\hat{y} - y)^2$.

Usando la **regla de la cadena**, escribe paso a paso las derivadas necesarias para obtener:

Gradiente respecto al peso w_1 :

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}.$$

Gradiente respecto al peso w_2 :

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_2}.$$

Gradiente respecto al sesgo b :

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b}.$$

Paso 5: Actualiza los parámetros con gradiente descendente

Con tasa de aprendizaje η :

$$w_1 \leftarrow w_1 - \eta \frac{\partial J}{\partial w_1},$$

$$w_2 \leftarrow w_2 - \eta \frac{\partial J}{\partial w_2},$$

$$b \leftarrow b - \eta \frac{\partial J}{\partial b}.$$

Ejercicio2: Ajuste de parámetros para permeabilidades relativas usando el algoritmo de Evolutivo Diferencial

En ingeniería de yacimientos, las permeabilidades relativas (k_r) suelen modelarse como funciones no lineales de la saturación de agua (S_w). Un modelo común es la función de Corey:

$$k_{rw} = k_{rw0} \left(\frac{S_w - S_{wr}}{1 - S_{wr} - S_{or}} \right)^{n_w}, \quad k_{ro} = k_{ro0} \left(\frac{1 - S_w - S_{or}}{1 - S_{wr} - S_{or}} \right)^{n_o} \quad (1)$$

Donde:

- k_{rw0} y k_{ro0} son las permeabilidades relativas máximas para agua y aceite.
- S_{wr} es la saturación residual de agua.
- S_{or} es la saturación residual de aceite.
- n_w y n_o son exponentes que determinan la forma de la curva.

Datos experimentales

Objetivo

Ajustar los parámetros del modelo de Corey para que las curvas generadas se ajusten a los datos dados, utilizando el algoritmo de **Evolución Diferencial** mediante la función `scipy.optimize.differential_evolution`.

S_w	k_{rw}	k_{ro}
0.20	0.02043	0.47018
0.25	0.03725	0.34831
0.30	0.06070	0.25510
0.35	0.09138	0.17839
0.40	0.12527	0.12192
0.45	0.15629	0.07397
0.50	0.20728	0.04302
0.55	0.25201	0.02135
0.60	0.30686	0.008581
0.65	0.37060	0.002643
0.70	0.43427	0.000306
0.75	0.51454	0.000000

Función Objetivo

La función objetivo, llamada `objetivo`, debe:

1. Recibir un vector de parámetros:

$$[k_{rw0}, k_{ro0}, S_{wr}, S_{or}, n_w, n_o] \quad (2)$$

2. Calcular las curvas modeladas de k_{rw} y k_{ro} para los valores de saturación S_w usando las ecuaciones de Corey.
3. Calcular el error cuadrático medio (MSE) entre los valores modelados y los experimentales:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left[\left(k_{rw, \text{modelo}}^{(i)} - k_{rw, \text{datos}}^{(i)} \right)^2 + \left(k_{ro, \text{modelo}}^{(i)} - k_{ro, \text{datos}}^{(i)} \right)^2 \right] \quad (3)$$

Donde:

- N es el número de puntos de saturación.
 - $k_{rw, \text{modelo}}^{(i)}$ y $k_{ro, \text{modelo}}^{(i)}$ son los valores generados por el modelo.
 - $k_{rw, \text{datos}}^{(i)}$ y $k_{ro, \text{datos}}^{(i)}$ son los valores experimentales.
4. Devolver el valor numérico del error.

Límites para los parámetros

Los rangos usados para los parámetros en el algoritmo de optimización son:

Parámetro	Rango
k_{rw0}	[0, 1]
k_{ro0}	[0, 1]
S_{wr}	[0, 1]
S_{or}	[0, 1]
n_w	[1, 5]
n_o	[1, 5]

$$\text{bounds} = [(0, 1), (0, 1), (0, 1), (0, 1), (1, 5), (1, 5)]$$

Aplicar Evolución Diferencial

Se usa el siguiente comando para encontrar los parámetros óptimos:

```
from scipy.optimize import differential_evolution
resultado = differential_evolution(objetivo, bounds)
```

El vector `resultado.x` contiene los parámetros óptimos encontrados.

Visualización de resultados

Una vez obtenidos los parámetros óptimos, grafica las curvas generadas por el modelo junto con los datos experimentales para evaluar visualmente la calidad del ajuste.

Documentación oficial de `differential_evolution`:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html